PLN as a Common Substrate for Naive Bayes and k-NN Premise Selection

Mettapedia Working Notes

February 19, 2026

Abstract

We present an evidence-primary premise-selection calculus in PLN style: revision ($\oplus$) for evidence accumulation, tensor ($\otimes$) for sequential update, and explicit normalization for evidence-budget control. Classical selectors are represented as specializations on this same carrier: NB as tensorized likelihood evidence (formal bridge in Lean), and k-NN as local evidence aggregation (formal bridge to MaSh-style relevance). We state all log-odds/pooling claims under explicit domain hypotheses: finite nonzero channels for odds/log-odds theorems, and nonzero finite totals for normalization/convex-mixture theorems. We also use the corrected pooling result: hplus is unique under external Bayesianity *plus* total-additivity, with `max` as the explicit counterexample when total-additivity is omitted. On extended MPTP 5k validation, round-2 (E, 5s, no `-p`) yields 290/800 for MaSh NB and 288/800 for the strongest PLN model (mixture-local Prior$\otimes$NB). Round-4 (E+Vampire, 28 runs) has mixture-local as the best single route on E (288/800) and best/tied-best on Vampire (174/800 at top-256, 173/800 at top-512). In targeted 30s top-256 follow-up, MaSh NB and mixture-local tie on E (327/800), with a small Vampire gap (184/800 vs 182/800).

1 Scope and Commitments

This paper addresses one concrete question: *which PLN rules are actually used in our premise selectors, how they compose, and under what formal hypotheses the key equivalences hold.*

There are two code layers:

- **Rule layer**: `hyperon/PeTTa/lib/lib_pln_xi.metta` (xiPLN wrappers over the nuPLN evidence algebra, plus calibrated aliases).

- **Selector layer**: PeTTa selectors in `hyperon/PeTTa/demos/pln_*_selector*.metta` and one pure-Python selector `atps/scripts/select_pln_mixture_local_nb.py`.

Methodologically, we stay in the Goertzel/Nil evidence-first discipline: the evidence object is primary, probabilities are projections, and composition is governed by algebraic laws rather than post-hoc score blending.

2 xiPLN Rule Kernel (lib_pln_xi)

For discrete evidence, xiPLN uses the same carrier as nuPLN:

$$E = (n^+, n^-), \quad E_1 \oplus E_2 = (n_1^+ + n_2^+,\, n_1^- + n_2^-),$$

$$E_1 \otimes E_2 = (n_1^+ n_2^+,\, n_1^- n_2^-), \quad \mathrm{power}(E, w) = (n^{+\,w}, n^{-\,w}).$$

Interpretation used throughout this paper: E is an unnormalized two-channel measure over $\{+, -\}$. Revision ($\oplus$) adds evidence mass, tensor ($\otimes$) multiplies channel-wise update factors, and strength/confidence are projections for ranking and reliability.

Operational views:

$$\mathrm{strength}(E) = \frac{n^+}{n^+ + n^-}, \qquad \mathrm{confidence}(E; \kappa) = \frac{n^+ + n^-}{n^+ + n^- + \kappa}.$$

Why include `power`? In this work, $\mathrm{power}(E, w)$ is used as an explicit *regraduation/calibration* operator in evidence space (for IDF, τ-gates, and prior/likelihood weighting), implemented in `hyperon/PeTTa/pln_inference/pln_evidence.metta`. It is forwarded unchanged by `nuPLN.power` and `xiPLN.discrete.power`. This is not a legacy `Truth_*` rule from `lib_pln.metta`; it is a calibrated extension layer over the same $E = (n^+, n^-)$ carrier. Its mathematical role is justified by Lean theorems `Evidence.toOdds_power_rpow` and `Evidence.toLogOdds_power_mul` (`Mettapedia/Logic/EvidenceQuantale.lean`):

$$\log\mathrm{odds}(\mathrm{power}(E, w)) = w \cdot \log\mathrm{odds}(E).$$

So w scales evidential impact linearly in log-odds space while keeping all composition in the evidence algebra ($\oplus, \otimes$), rather than as ad-hoc post-hoc score hacking. This is continuous with the PLN-book scalar idea (scale evidence mass while preserving the underlying evidential meaning), but different in mechanism: book-style scalar acts on total-count weight (the (s, N)

Rule name (`lib_pln_xi.metta`)	**Definition / role**
`ContextualPriorRevision`	STV-level revision of global and local prior ($\oplus$).
`PriorLikelihoodTensorSTV`	Tensor-compose prior and likelihood evidence, then project to STV.
`NormalizedPriorLikelihoodTensor`	Regrade prior/likelihood via `power`, then tensor-compose and project.
`WeakNegativeSafeUpdate`	Add weak negative evidence by revision: $E \mapsto E \oplus (0, \epsilon)$.
`RegraduatedEvidence`	Explicit evidence regraduation alias: $E \mapsto \mathrm{power}(E, w)$.
`TensorStrengthEqNBPosterior`	STV wrapper exposing the NB bridge at the rule API boundary.

Table 1: xiPLN rule surface used for selector composition. All rules are in the `xiPLN.Rule.*` or `xiPLN.Bridge.*` namespace.

view), whereas power acts directly on the (n^+, n^-) carrier to produce linear log-odds regraduation.

Semantic contract. xiPLN is an API/calibration facade over nuPLN, not a new calculus: the discrete carrier and core composition laws are identical $(E, \oplus, \otimes)$; xiPLN contributes naming, default calibration parameters, and task-facing wrappers only.

Layer	**Provenance / semantic status**
Legacy PLN rules	`Truth_*` rules in `hyperon/PeTTa/lib/lib_pln.metta`, with direct PLN-book grounding (e.g., `Truth_Revision`, book §5.10.2).
Evidence-calibration extension	`evidence-hplus`, `evidence-tensor`, `evidence-power` in `pln_inference/pln_evidence.metta`, wrapped by `nuPLN/xiPLN`; coherence justified by Lean evidence theorems listed in Section 6.

Table 2: Legacy-vs-extension provenance for the rules used in this paper.

2.1 Rule list (precise mathematical form)

For a goal g, candidate premise a, and evidence values $E = (n^+, n^-)$, $F = (m^+, m^-) \in \mathbb{R}^\infty_{\geq 0} \times \mathbb{R}^\infty_{\geq 0}$:

$$
\begin{aligned}
\textbf{R1 (Revision)} \quad & E \oplus F := (n^+ + m^+,\ n^- + m^-), \\
\textbf{R2 (Tensor update)} \quad & E \otimes F := (n^+ m^+,\ n^- m^-), \\
\textbf{R3 (Strength projection)} \quad & s(E) := \frac{n^+}{n^+ + n^-}, \\
\textbf{R4 (Confidence projection)} \quad & c_\kappa(E) := \frac{n^+ + n^-}{n^+ + n^- + \kappa}, \\
\textbf{R5 (Regraduation)} \quad & \mathrm{pow}_w(E) := (n^{+\,w},\ n^{-\,w}), \\
\textbf{R6 (Normalization)} \quad & \mathrm{Norm}_t(E) := \big(t\, s(E),\ t(1 - s(E))\big), \\
\textbf{R7 (Weak negative safety)} \quad & \mathrm{WeakNeg}_\epsilon(E) := E \oplus (0, \epsilon), \\
\textbf{R8 (Staged selector posterior)} \quad & E_{\mathrm{post}}(g, a) := \Big(\bigoplus_i \mathrm{Norm}_{t_i(g)}(\mathrm{pow}_{w_i}(E_i(g, a))) \Big) \\
& \qquad \otimes \mathrm{Norm}_{t_L}(E_{\mathrm{like}}(g, a)).
\end{aligned}
$$

Ranking is then performed by $\mathrm{score}(g, a) := s(E_{\mathrm{post}}(g, a))$, with optional confidence-aware tie-breaking via c_κ.
These are not free-floating definitions: they are tied to Lean contracts listed in Section 6, including log-odds additivity under tensor, linear log-odds scaling under regraduation, normalization-preserves-strength, and staged commutation laws.

2.2 Formal domain assumptions (explicit)

Claims such as "log-pool equivalence" or "additivity in log-odds" are *not* unconditional in our Lean development; they are theorem-level claims under explicit hypotheses.

- **Tensor in odds/log-odds**: `Evidence.toOdds_tensor_mul` and `Evidence.toLogOdds_tensor_add` require finite/nonzero regime assumptions (notably nonzero negative channels and finite/nonzero odds).
- **Regraduation in log-odds**: `Evidence.toLogOdds_power_mul` requires $w \geq 0$, $n^- \neq 0$, and finite/nonzero odds.

Rule	Lean contract	Implementation locus
R1 ($\oplus$, revision)	`PLN_revisionStrength_eq_linearPool`	`pln_inference/pln_evidence.metta`, `xiPLN.Rule.ContextualPriorRevision`
R2 ($\otimes$, update)	`Evidence.toLogOdds_tensor_add`	`pln_inference/pln_evidence.metta`, `xiPLN.Rule.PriorLikelihoodTensorSTV`
R5 (pow, regraduation)	`Evidence.toLogOdds_power_mul`	`pln_inference/pln_evidence.metta`, `xiPLN.Rule.RegraduatedEvidence`
R6 (Norm, mass control)	`normalizeEvidence_toStrength`	`Mettapedia/Logic/PremiseSelectionFusion.lean`, normalized selector stages
R8 (staged posterior map)	`stagedFamilyPosterior_commute`, `stagedFamilyPosterior_regrade_roundtrip_commute`	`atps/scripts/select_pln_mixture_local_nb.py`

Table 3: Compact rule→theorem→implementation crosswalk.

- **Normalization contracts**: `normalizeEvidence_toStrength` and `fuse_toStrength_normalized_totals` require $t \neq 0$ and $t \neq \top$ (finite nonzero evidence totals).

Accordingly, all selector equations in this paper are understood in that same finite/nonzero regime unless stated otherwise. When we say "log-pool" in this paper, we mean this precise theorem-backed statement: channel-wise tensor update induces additive log-odds under the hypotheses of `Evidence.toLogOdds_tensor_add`.

3 How Rules Are Composed in Our Selectors

3.1 IDF-NB selector (likelihood-heavy baseline)

Files: `hyperon/PeTTa/demos/pln_idf_nb_selector_jobs.metta`, `atps/scripts/select_pln_nb.py`.

Construction:

- Prior evidence per axiom from tfreq table.
- Feature evidence from `lookup_feature_idf_db` with IDF weights.
- Log-domain fold over features: $\log P^+, \log P^-$ accumulated with weighted $\log n^+, \log n^-$.

- STV projection: strength $= \sigma(\log P^+ - \log P^-)$, confidence from a log-total vs $\log \kappa$ gate.

3.2 kNN-prior + NB-likelihood (PeTTa merged model)

Files: `hyperon/PeTTa/demos/pln_knn_prior_nb_selector_opt.metta`, `atps/scripts/select_pln_knn_prior_nb_opt.py`.

Construction:

- Per-neighbor overlap (`goal_feat_db` $\cap$ `nbr_feat_db`) computed in Prolog backend.
- Canonical evidence transfer is kNN-positive / weak-negative aggregation: used neighbors contribute $(w, 0)$, non-used neighbors contribute $(0, \eta w)$, then rows are aggregated by `evidence-hplus`.
- The legacy PeTTa implementation includes a modus-ponens-style row transfer in `pln_rules.metta`; in this paper, the semantic contract is the evidence-level transfer above (which is what the Lean kNN bridge captures).
- Goal prior is revised: $E_{\text{prior}} = E_{\text{global}} \oplus E_{\text{knn}}$.
- NB likelihood (features only) is composed in log-domain with prior odds to produce final STV.

Legacy MP vs canonical transfer. The PLN book treats probabilistic MP as useful but heuristic/error-prone. For premise selection we therefore take evidence-level kNN transfer ($(w, 0)$ for used, $(0, \eta w)$ for weak-negative) as canonical semantics, and treat MP-style row transfer as a legacy implementation detail.

3.3 Mixture-local Prior$\otimes$NB (staged model)

File: `atps/scripts/select_pln_mixture_local_nb.py`.

This is the explicit staged architecture:

- **Stage A**: build multiple local experts from neighborhood bins $[4, 16, 64, 256]$, with weak negatives.
- **Stage B**: normalize each expert to target mass and pool via evidence addition ($\oplus$).

- **Stage C**: tensor-compose pooled prior with likelihood-only NB evidence ($\otimes$).

Formally:

$$E_{\text{post}}(g,a) = \Big(\bigoplus_i \text{Normalize}_{t_i(g)}(E_i^{\text{local}}(g,a)) \oplus \text{Normalize}_{t_G}(E^{\text{global}}(a))\Big) \otimes \text{Normalize}_{t_L}(E^{\text{NB-like}}(g,a)).$$

The staged map commutes in the external-Bayes sense and is stable under regrade/unregrade round-trip: `stagedFamilyPosterior_commute` and `stagedFamilyPosterior_regrade_roundtrip_commute` (`Mettapedia/Logic/PremiseSelectionExternalBayesianity.lean`).

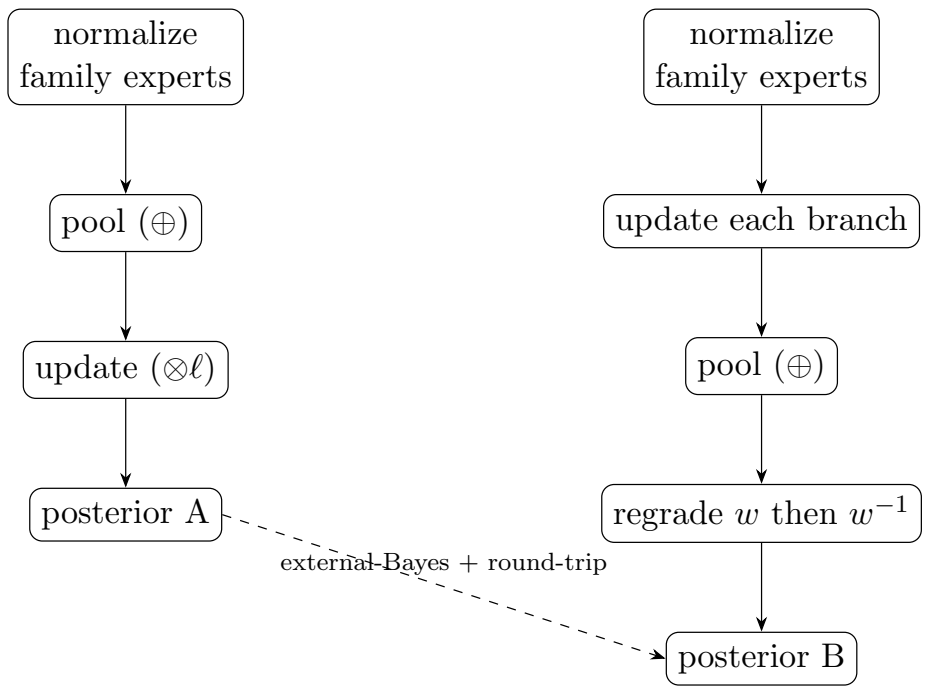

Figure 1: Commuting staged selector diagram for the normalized family pool/update pipeline with regrade round-trip coherence.

3.4 Unified selector derivation skeleton

All selectors in this paper instantiate the same calculus template:

$$
\begin{aligned}
&E^{(i)}_{\text{prior}}(g,a) \xrightarrow{\text{power}} \widetilde{E}^{(i)}_{\text{prior}}(g,a) \xrightarrow{\text{Normalize}_{t_i(g)}} \widehat{E}^{(i)}_{\text{prior}}(g,a),\\
&E_{\text{pool}}(g,a) = \bigoplus_i \widehat{E}^{(i)}_{\text{prior}}(g,a),\\
&E_{\text{post}}(g,a) = E_{\text{pool}}(g,a) \otimes \text{Normalize}_{t_L}(E_{\text{like}}(g,a))\,,\\
&\text{score}(g,a) = \text{strength}(E_{\text{post}}(g,a))\,.
\end{aligned}
$$

NB-only, kNN+NB, and mixture-local models differ only in how $E^{(i)}_{\text{prior}}$, t_i, and E_{like} are instantiated. The operator laws are shared.

4 Lean Contracts for the Rule Stack

The implementation-level rules above are tied to explicit Lean contracts. Abbreviation: `EB` = `PremiseSelectionExternalBayesianity.lean`.

Core bridges.

- **NB bridge**: `toStrength_nbEvidence_eq_nbPosterior` (`PLNBayesNetInference.lean`).
- **k-NN bridge**: `PLN_hplusPos_eq_knnRelevance` (`PremiseSelectionKNN_PLNBridge.lean`).
- **Revision = linear pooling**: `PLN_revisionStrength_eq_linearPool` (`PremiseSelectionFusion.lean`).
- **Prior/NB two-stage**: `priorNBPosterior_eq_twoStage` (`PremiseSelectionPriorNB.lean`).

Evidence-algebra laws (in `EvidenceQuantale.lean`). Regraduation: `Evidence.toLogOdds_power_mul`, `Evidence.toOdds_power_rpow`. Round-trip canaries: `Evidence.power_power_inv`, `Evidence.toLogOdds_power_power_inv`.

Normalization and fusion (in `PremiseSelectionFusion.lean`). `normalizeEvidence_toStrength` (preserves strength for $t \neq 0$, $t \neq \top$); `fuse_toStrength_normalized_totals` (explicit convex weights).

Pooling uniqueness (in `EB`). `poolE_eq_hplus_of_externalBayes_totalAdd` (hplus is unique under external Bayesianity + total-additivity), plus counterexample `maxPoolingOperator_ne_fuse`.

Regrading and staged coherence (in `EB`). `toLogOdds_scaleEvidence` (regrading invariance); `externalBayesianity_fuseFamily_tensor` (family-level commutation); `stagedFamilyPosterior_commute` and `stagedFamilyPosterior_regrade_roundtrip_commute` (staged-pipeline canaries).

Additional contracts. `finite_statistic_tv_mixture_bound_m16_R4551` (`PremiseSelectionLocalMixtureBridge.lean`); `fusion_ranking_after_normalization_toReal` (`PremiseSelectionOptimality.lean`); OSLF modal corollaries `oslf_diamond_stagedFamily_roundtrip` and `oslf_box_stagedFamily_roundtrip` (`OSLF/Framework/PLNSelectorGSLT.lean`).
For a selector-level composition theorem that mirrors the staged architecture, see `Mettapedia/Logic/PremiseSelectionBestPLNDraft.lean`.

5 Experimental validation on extended MPTP 5k (rounds 1–4)

5.1 Protocol and data integrity

We evaluate seven selectors under one canonical protocol:

- Dataset: 4200 train problems, 800 validation problems.
- Model-generation data source: pooled dependency events[1] (`pooled_train_deps_allruns.jsonl`), built from base train deps plus prior train proof runs; final pool size is 9045 events over 1649 unique training problems.

[1] In evidential semantics, a "round" is a data-provenance epoch (new proof observations), not a new logical calculus. Pooling rounds means accumulating additional evidence counts via revision ($\oplus$); normalization keeps evidence budgets explicit before tensor composition ($\otimes$). Thus round labels record chronology/integrity of evidence acquisition, while core PLN operators remain fixed.

- Selection budgets: top-256 and top-512.
- Prover route: E 3.0.1-ho, timeout 5s, jobs=4, *without* E's `-p`.
- Integrity policy: generate top-512 selections first, derive top-256 by prefix truncation, then run E on both budgets.

We report: (i) round-1 reference validation, (ii) round-2 pooled-data validation, and (iii) round-4 cross-prover validation. Training *rounds* refer to provenance epochs for building pooled training evidence, while validation *runs* are fixed-budget prover executions on the held-out 800-problem split using selectors trained from that pooled evidence.

5.2 Methods in PLN rule style

In the PLN book spirit, we keep evidence as the primary object and use only: revision ($\oplus$, evidence accumulation), tensor ($\otimes$, sequential composition), and normalization (fix evidence mass while preserving strength). Across training rounds, we accumulate dependency evidence by revision:

$$E_{\text{pool}}(a) = \bigoplus_{r \in \mathcal{R}} E_r(a),$$

where $E_r(a)$ is the evidence contribution for premise a from round r. Thus, adding rounds means adding evidence mass in the same algebra; it does not change the selector logic or operator semantics.

For the new mixture-local selector (`select_pln_mixture_local_nb.py`), the staged construction is:

$$E_{\text{post}}(g, a) = \Big(\bigoplus_i \text{Normalize}_{t_i(g)}(E_i^{\text{local}}(g, a)) \oplus \text{Normalize}_{t_G}(E^{\text{global}}(a)) \Big) \otimes \text{Normalize}_{t_L}(E^{\text{NB-like}}(g, a)).$$

Here each local expert i is built from a k-NN bin:

$$n_i^+(g, a) = \sum_{\chi \in B_i(g)} \text{sim}(g, \chi)\, \mathbf{1}[a \in \mathit{deps}(\chi)],$$

$$n_i^-(g, a) = \eta \sum_{\chi \in B_i(g)} \text{sim}(g, \chi)\, \mathbf{1}[a \notin \mathit{deps}(\chi)].$$

We use $E_i^{\text{local}} = (n_i^+ + \alpha,\ n_i^- + \beta)$, with weak-negative factor $\eta = 0.08$, bins $[4, 16, 64, 256]$, and similarity $\text{sim} = \text{overlap}/(\text{overlap} + \kappa)$. Likelihood

evidence is *NB-likelihood only* (prior removed by setting $\sigma_1 = 0$), then robustly scaled per problem (IQR target) before conversion to odds evidence and normalization.

This gives a non-alias architecture:

- Stage A: multiple local priors from neighbor bins.
- Stage B: normalized pooling via revision ($\oplus$).
- Stage C: normalized tensor with likelihood-only NB evidence.

What this claims (and does not). The Lean contracts certify algebraic coherence of the selector pipeline (pool/update/regrade/normalize laws under stated hypotheses). They do not prove that any one concrete script is globally optimal for ATP success under fixed time budgets. Empirical quality still depends on data quality, feature design, and calibration of local evidence reliability.

5.3 Round-1 reference results

To keep the chronology explicit, Table 4 lists the pre-pooled round-1 validation runs used before the round-2 pooled-data refresh.

Model (round1 reference)	Top-256	Top-512
MaSh NB	283/800 (35.38%)	291/800 (36.38%)
PLN kNN-prior-NB (opt, merge-NB)	282/800 (35.25%)	291/800 (36.38%)
PLN NB	281/800 (35.12%)	n/a
PLN Normal-NB	279/800 (34.88%)	n/a
PLN Enhanced	276/800 (34.50%)	n/a
MaSh k-NN	272/800 (34.00%)	291/800 (36.38%)

Table 4: Round-1 validation reference results (legacy pre-pooled run set).

5.4 Validation results (canonical 14 runs)

Table 5 reports all seven models at top-256 and top-512.

5.5 Round-2 discussion

- **Top-256 precision regime**: MaSh NB is best (290), but the new PLN mixture-local model is very close (288). The margin is 2/800.
- **Top-512 recall regime**: MaSh k-NN and PLN mixture-local tie for best (286), with MaSh NB and two PLN variants at 285. At this budget, differences are effectively tie-level.

Model (round2_pooled)	Top-256	Top-512
MaSh NB	290/800 (36.25%)	285/800 (35.63%)
PLN mixture-local Prior⊗NB	288/800 (36.00%)	286/800 (35.75%)
PLN kNN-prior-NB (opt, merge-NB)	281/800 (35.12%)	285/800 (35.63%)
MaSh k-NN	280/800 (35.00%)	286/800 (35.75%)
PLN Normal-NB	280/800 (35.00%)	282/800 (35.25%)
PLN NB	276/800 (34.50%)	285/800 (35.63%)
PLN Enhanced	272/800 (34.00%)	283/800 (35.38%)

Table 5: Round-2 validation results (E5s, no-`-p`, jobs=4).

- **Overlap is high**: at top-256, MaSh NB and PLN mixture-local share 287 solved problems, with only 3 NB-unique and 1 mixture-unique. At top-512, MaSh NB vs MaSh k-NN differs on just 5 problems total.
- **Greedy-cover complementarity remains small but real**: top-5 greedy union is 297 solved with all runs, and 296 without MaSh runs. So MaSh adds one extra unique solve in this top-5 cover view.
- **PLN interpretation**: these results match the formal story. Revision and tensor composition are structurally correct, but empirical gains depend on calibration quality of local priors and gating, not on operator choice.

5.6 Round-4 cross-prover validation (canonical 28 runs)

Round-4 evaluates seven selector tags across two prover routes (E no-`-p` and Vampire), each at top-256 and top-512:

$$7 \times 2 \times 2 = 28 \text{ runs.}$$

Protocol: validation split (800 problems), timeout 5s, jobs=6 for both provers. The compact artifact is therefore named `results_round4_val_5s_compact_28runs.csv`. Separate targeted top-256 30s runs were executed later for the two strongest models only.

5.7 Round-4 discussion

- **Best single run**: PLN mixture-local Prior⊗NB is strongest on E (288/800) and also best on Vampire at top-256 (174/800), tying best at top-512 Vampire (173/800).
- **Top-256 vs top-512 behavior**: for the winning model, E@256 and E@512 are identical (288), indicating ranking saturation by 256 for this setup.

Model (round4 tags)	E@256	E@512	V@256	V@512
MaSh NB (E-trained)	284	277	171	172
MaSh NB (V-trained)	281	280	170	169
MaSh k-NN (E-trained)	278	285	171	171
MaSh k-NN (V-trained)	277	283	170	170
PLN kNN-prior-NB+MaSh (E-trained)	276	280	170	173
PLN kNN-prior-NB+MaSh (V-trained)	280	280	172	173
PLN mixture-local Prior⊗NB (EV-pooled)	**288**	**288**	**174**	**173**

Table 6: Round-4 validation results (E no-`-p` and Vampire, 5s, jobs=6).

- **Vampire spread is compressed**: all Vampire runs lie in a narrow band (169–174 solved), suggesting lower sensitivity to selector differences at 5s.
- **Cross-route complementarity is real**: greedy cover over all 28 runs reaches 321/800, while the top-5 greedy picks already reach 320/800.
- **MaSh contribution in cover space**: greedy cover without MaSh-tagged runs is 315/800; including MaSh raises the full cover to 321/800.

5.8 Targeted 30s follow-up (top-256)

To test whether 5s outcomes were dominated by prover-time limits, we ran a targeted 30s follow-up on the two strongest models from round-4: MaSh NB (E-trained) and PLN mixture-local Prior⊗NB (EV-pooled), both at top-256 on the same 800 validation problems.

Model (top-256, 30s)	E no-`-p`	Vampire
MaSh NB (round4_e)	327/800 (40.88%)	**184/800** (23.00%)
PLN mixt.-local Prior⊗NB (round4_ev)	327/800 (40.88%)	182/800 (22.75%)

Table 7: Targeted 30s validation follow-up at top-256.

Interpretation: at longer E time budgets, both models converge to the same solve count, while on Vampire the gap remains small (2 problems).

5.9 Implementation notes

- ATP drivers: `atps/scripts/select_pln_nb.py`, `atps/scripts/select_pln_normal_nb.py`, `atps/scripts/select_pln_enhanced.py`, `atps/scripts/select_pln_knn_prior_nb_opt.py`.
- PeTTa selector backends: `hyperon/PeTTa/demos/pln_*_selector*.metta`.
- Library usage: `hyperon/PeTTa/lib/lib_pln_nu.metta` and `hyperon/PeTTa/pln_inference/pln_evidence.metta/` `hyperon/PeTTa/pln_inference/pln_rules.metta` are the main paths for current discrete selectors; `hyperon/PeTTa/lib/lib_pln_xi.metta` is used explicitly in the normal-carrier path and as the rule-level API target for the xiPLN exposition in this paper.
- Pure-Python selector: `atps/scripts/select_pln_mixture_local_nb.py` (same evidence algebra, staged explicitly for auditability/performance).
- Round-4 queue/orchestration: `atps/scripts/run_round4_val_execute_and_analyze.sh`.
- Round-4 compact scoreboard and greedy artifacts: `results_round4_val_5s_compact_28runs.csv`, `greedy_cover_round4_val_all.csv`, `greedy_cover_round4_val_no_mash.csv`.

6 Lean map

All paths are relative to the project root. Abbreviation: `M/L/` = `Mettapedia/Logic/`.

Module	Contents
`M/L/EvidenceQuantale.lean`	Evidence carrier, semiring/quantale laws, odds/log-odds and power laws
`M/L/PLNCore.lean`	Curated top-level import surface for the theorem-complete PLN stack
`M/L/PremiseSelectionKNN.lean`	General k-NN definitions; MaSh instance
`M/L/PremiseSelectionKNN_PLNBridge.lean`	PLN–k-NN bridge; paper-form equivalence
`M/L/PLNBayesNetInference.lean`	PLN NB bridge; strength = NB posterior
`M/L/PremiseSelectionFusion.lean`	Fusion via revision; normalization
`M/L/PremiseSelectionOptimality.lean`	Optimality transfer lemmas (NB, k-NN, PLN, fusion, ranking)

Module	Contents
`M/L/PremiseSelectionExternalBayesianity.lean`	Counterexample + corrected uniqueness; regraduation invariance; staged commutation canaries
`M/L/PremiseSelectionOperatorRoles.lean`	Role discipline structures
`M/L/PremiseSelectionPriorNB.lean`	Prior-NB composition; canonical aliases
`M/L/PremiseSelectionLocalMixtureBridge.lean`	Finite local-mixture TV bounds
`M/L/DiaconisFreedmanFinite.lean`	Base collision bound
`M/L/PremiseSelectionCoverage.lean`	Submodular coverage
`M/L/PremiseSelectionPUCalibration.lean`	PU weak-negative calibration
`M/L/PremiseSelectionRankingStability.lean`	Ranking stability
`M/L/PremiseSelectionSelectorSpec.lean`	Selector spec defaults/checklist
`M/L/PremiseSelectionPartitionedPriorNB.lean`	Partitioned normalized Prior-NB
`hyperon/PeTTa/tests/test_lib_pln_xi_rules.metta`	Deterministic xiPLN rule tests